

《软件测试与维护》课程大作业报告

Online Boutique 微服务测试、论文复现 与端到端 AIOps 智能运维实践

项目	内容
微服务系统	Online Boutique
论文一	USAD：多变量时间序列无监督异常检测
论文二	KPIRoot：基于监控指标的根本定位
加分项	端到端实时 AIOps Agent 智能运维控制台
指导教师	张圣林
完成日期	2026 年 6 月 9 日

小组成员

姓名	学号	主要负责方向
李响	2311467	USAD 主体复现、系统部署与监控、自研微服务
陈祖名	2313163	ChaosMesh 真实闭环、JMeter 并发矩阵、Edge 兼容性，报告整理
王新杰	2311901	KPIRoot 论文复现、监控采集与故障数据整理
王子祺	2312385	KPIRoot 方案讨论、结果校核与报告整理
王鼎	2213002	AIOps 控制台、实时流水线与智能体集成

摘要

本项目面向云原生微服务系统的部署、测试与智能运维需求，选择 Online Boutique 作为实验对象，构建了覆盖环境部署、监控采集、故障注入、功能与性能测试、异常检测、根因定位和智能诊断的完整实验链路。项目在 Minikube 中部署 Online Boutique，并使用 ChaosMesh 制造可控故障，通过 Prometheus、HTTP 探针和 Kubernetes API 采集正常、故障与恢复阶段的多维 KPI；使用 Selenium 和 JMeter 验证核心购物流程、浏览器兼容性以及 10/30/50 并发下的性能变化。算法复现部分选择 KDD 2020 的 USAD 和 ISSRE 2024 的 KPIRoot：前者通过双解码器重构误差检测多变量时间序列异常，后者通过 PAA、SAX 相似度与 Granger 风格因果得分定位根因指标。USAD 在 120 点 ChaosMesh 真实闭环数据上取得 Precision 0.4643、Recall 0.7429、F1 0.5714；KPIRoot 在 paymentservice CPU Stress、frontend CPU Stress 和 paymentservice Pod Kill 三类场景中均将真实根因服务排在第一位。进一步地，项目实现端到端 AIOps 控制台，能够从 Prometheus 采集实时指标，生成 USAD/KPIRoot runtime 输入并真实执行算法，由 Agent 结合 Kubernetes 与 Prometheus 证据生成诊断报告，并可选调用火山方舟 Ark LLM 生成结构化总结。系统将恢复动作严格限制为 dry-run，实现了“真实注入、真实采集、真实检测与定位、人工确认恢复”的安全闭环。实验表明，两类算法在功能上具有明确互补性，能够形成从异常发现到根因解释的可演示、可复现、可扩展智能运维原型。

关键词：Online Boutique；Prometheus；ChaosMesh；Selenium；JMeter；USAD；KPIRoot；AIOps

目录

摘要 2

目录 3

1 项目概述与总体设计 5

 1.1 项目背景与目标 5

 1.2 课程要求与项目实现对照 5

 1.3 总体技术路线 5

2 Online Boutique 部署与微服务扩展 6

 2.1 系统选择与部署环境 6

 2.2 自研微服务与第三档目标 7

3 监控、故障注入与数据采集 7

 3.1 Prometheus 与 Grafana 监控体系 7

 3.2 故障场景设计 8

 3.3 数据组织与质量控制 9

4 Selenium 与 JMeter 黑盒测试 10

 4.1 Selenium 功能与兼容性测试 10

 4.2 JMeter 多并发性能测试 11

5 论文一：USAD 异常检测复现 12

 5.1 方法原理 12

 5.2 三层实验设计与结果 12

 5.3 结果解释与局限 13

6 论文二：KPIRoot 根因定位复现 13

 6.1 问题定义与核心算法 13

 6.2 Online Boutique 场景适配 14

 6.3 主实验结果 14

 6.4 消融实验 15

7 端到端实时 AIOps 智能运维 16

- 7.1 建设目标与系统架构 16
- 7.2 推荐演示主线 17
- 7.3 实时流水线运行证据 18
- 7.4 Agent 与 Ark LLM 诊断 19
- 7.5 安全边界 20
- 8 两篇算法与 AIOps 的综合分析 21
 - 8.1 USAD 与 KPIRoot 的互补关系 21
 - 8.2 结果一致性与冲突处理 21
- 9 达成度、创新点与不足 22
 - 9.1 课程目标达成度 22
 - 9.2 项目创新点 22
 - 9.3 局限与改进方向 22
- 10 小组成员分工 23
- 11 总结 23
- 参考文献 24
 - 项目仓库 24
- 附录 A 关键运行命令 25
- 附录 B 报告证据索引 25

目录将在打开文档时自动更新。

1 项目概述与总体设计

1.1 项目背景与目标

课程大作业要求围绕微服务系统完成部署、监控维护、黑盒测试、故障注入、异常数据采集以及两篇论文算法复现，并鼓励在更复杂的开源微服务系统上开展微服务开发和智能体封装。本组没有停留在 SockShop 指南的基础部署层面，而是选择服务数量更多、调用链更完整的 Online Boutique，围绕“可观测、可注入、可测试、可检测、可定位、可诊断”六项能力设计统一实验。项目不仅给出单项工具结果，还强调阶段之间的数据连续性：ChaosMesh 制造可控故障，Prometheus 采集故障窗口数据，USAD 判断是否异常，KPIRoot 定位异常来源，Agent 综合在线证据形成可执行建议。

总体目标：将课程四个阶段组织为一个闭环工程，而不是相互独立的工具截图集合；所有关键结论均由源码、CSV、图表或运行报告支撑。

1.2 课程要求与项目实现对照

课程阶段	核心要求	本项目实现	证据
阶段一	部署微服务系统并阅读两篇论文	Minikube 部署 Online Boutique；选取 USAD 与 KPIRoot	Kubernetes 清单、论文及复现源码
阶段二	Prometheus/Grafana 监控与 ChaosMesh 故障注入	采集 CPU、内存、应用探针和 Pod 状态；构造 CPU Stress、Pod Kill 等场景	监控截图、原始查询、metadata、KPI CSV
阶段三	Selenium 功能测试与 JMeter 性能测试	核心购物路径自动化；Edge 兼容性；10/30/50 并发矩阵	截图、JSON、JTL、汇总表
阶段四	使用采集数据运行论文算法	真实 KPI 输入 USAD；三场景输入 KPIRoot；输出检测与排序结果	anomaly_scores、ranking、summary、消融表
第三档	复杂系统基础上开发 1—2 个微服务	anomaly-detector 与 online-boutique-probe-exporter	Dockerfile、Deployment、Service、接口证据
加分项	智能体自动进行智能运维	实时采集→USAD→KPIRoot→Agent→可选 LLM→报告中心	Dashboard、runtime report、Agent/LLM 报告

1.3 总体技术路线

总体技术路线分为离线可控复现、真实环境验证和实时智能运维三层。第一层使用可标注 KPI 验证算法实现是否正确；第二层将算法迁移到 Online Boutique 的真实故障数据，检验部署、采集和

算法链路；第三层由 AIOps 控制台按本次故障重新采集 Prometheus 数据并执行算法，使诊断结果与当前实验相对应。三层设计兼顾算法评价的可控性、工程环境的真实性和端到端运行的完整性。

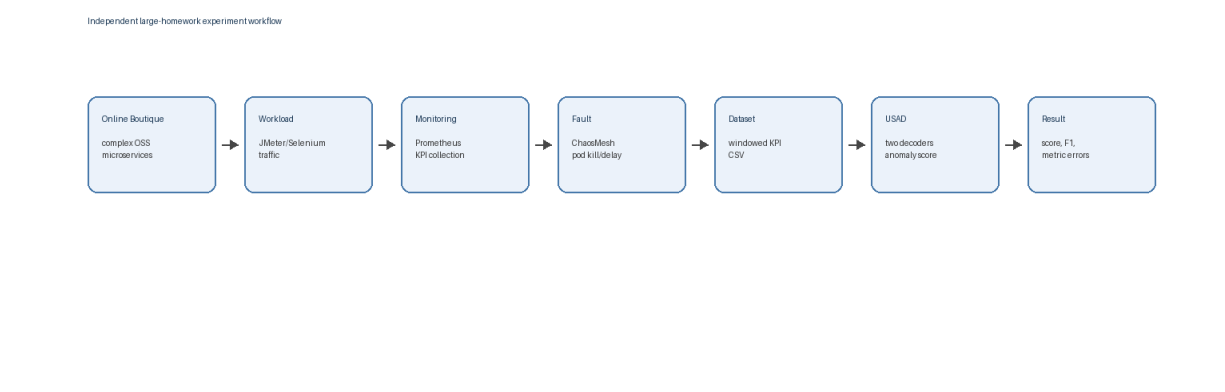


图 1 项目从数据生成、故障注入到 USAD 检测的基础实验流程

- 异常检测层：USAD 判断多变量 KPI 窗口是否偏离正常模式，并输出异常分数与指标级重构误差。
- 根因定位层：KPIRoot 在告警 KPI 已异常的前提下，对候选 KPI 进行相似度、因果性和综合得分排序。
- 证据融合层：Agent 将算法输出与 Kubernetes、Prometheus 当前状态合并，生成风险等级和恢复建议。
- 安全控制层：故障注入、数据采集和算法运行可真实执行；恢复命令仅以 dry-run 草案形式输出。

2 Online Boutique 部署与微服务扩展

2.1 系统选择与部署环境

Online Boutique 是由多个语言和框架实现的云原生电商示例，包含 frontend、cartservice、productcatalogservice、currencyservice、paymentservice、shippingservice 等核心服务。相较 SockShop，该系统服务数量更多、业务流程更长，适合观察资源、可用性与前端体验之间的关联。实验环境采用 Windows、Docker Desktop、Minikube、kubectl 和 online-boutique 命名空间。部署完成后核心 Pod 均处于 Ready 状态，前端通过 port-forward 暴露并可正常返回 HTTP 200。

组件	作用	实验中的使用方式
Docker Desktop	容器运行时	运行 Minikube 及相关镜像
Minikube/Kubernete s	微服务编排	部署 Online Boutique、监控组件和自研服务
Prometheus	指标采集	采集容器、Pod、Deployment 与应用探针指标
Grafana	监控可视化	观察基线、故障和恢复阶段变化

组件	作用	实验中的使用方式
ChaosMesh	故障注入	注入 CPU Stress、Pod Kill 等可控故障

2.2 自研微服务与第三档目标

为满足第三档“在复杂开源微服务系统基础上开发一到两个微服务”的要求，项目新增 anomaly-detector 和 online-boutique-probe-exporter。anomaly-detector 将 USAD 从离线脚本封装为可部署接口，通过 /health 验证服务状态，通过 /detect 读取数据并返回 Precision、Recall、F1 等检测摘要；probe-exporter 通过主动访问 Online Boutique 前端，将请求数、响应时间、错误数、状态码和响应字节数暴露为 Prometheus 指标。两个服务均提供 Dockerfile、Kubernetes Deployment、Service 和健康检查配置，使算法能力和应用层观测能力能够进入集群运行。

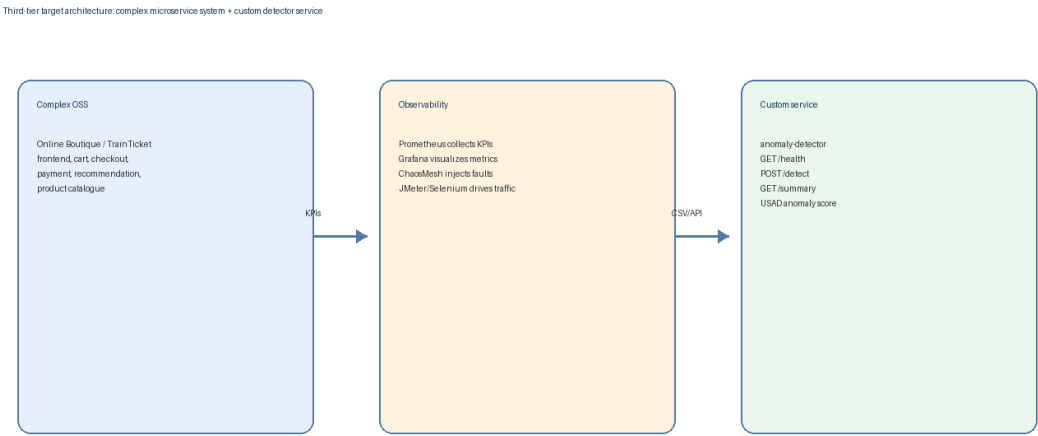


图 2 第三档目标架构：Online Boutique、自研监控探针与异常检测服务

工程价值：自研服务不是为了替换原系统业务，而是补充可观测性与算法服务化能力，体现“微服务开发与运维流程集成”。

3 监控、故障注入与数据采集

3.1 Prometheus 与 Grafana 监控体系

监控体系同时覆盖资源层、编排层和应用层。资源层包括容器 CPU、内存和文件系统读写；编排层包括 Pod phase、restart count、Deployment ready replicas；应用层由 probe-exporter

提供前端请求次数、时延、错误数、状态码和响应字节数。Grafana 面板用于在同一时间轴上对比基线、故障与恢复阶段，为算法输入筛选和结果解释提供直观证据。

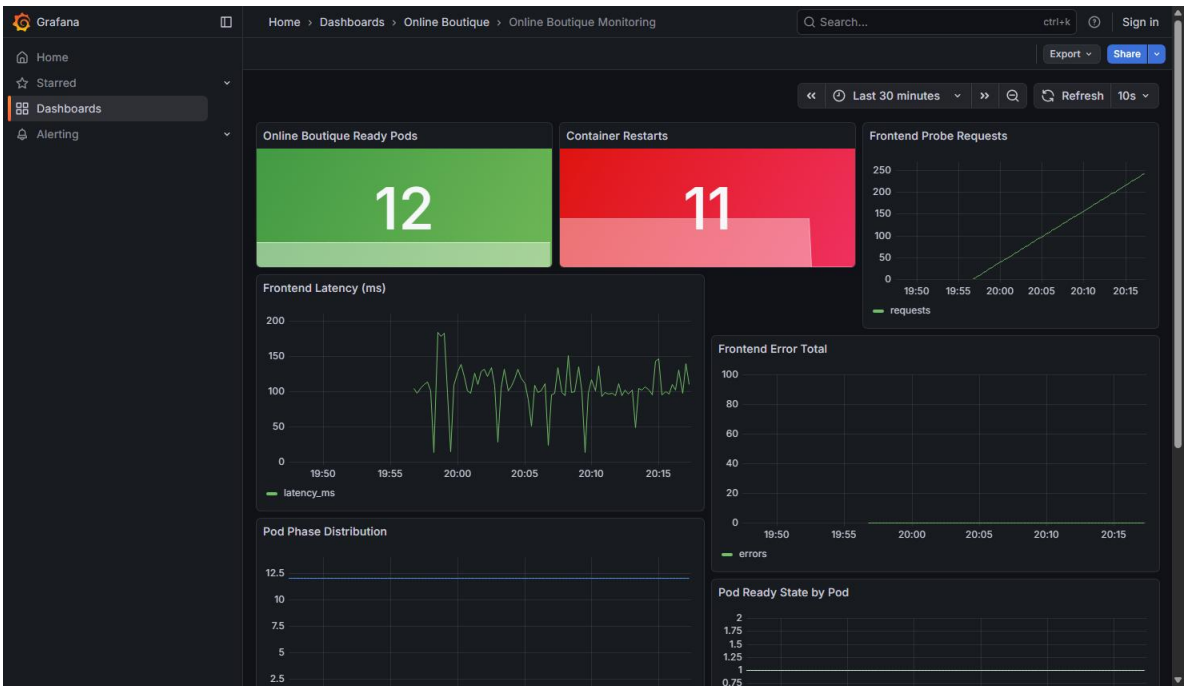


图 3 Online Boutique 应用性能与 Kubernetes 状态监控面板

3.2 故障场景设计

场景	目标服务	注入方式	主要预期表现	用途
PodChaos	productcatalogservice	杀死目标 Pod 后由 Deployment 重建	Ready 比例、响应状态和时延波动	USAD 120 点真实闭环
CPU Stress	paymentservice	StressChaos, 80% CPU, 1 worker	service_cpu_rate 超阈值	KPIRoot 与 AIOps 主演示
CPU Stress	frontend	StressChaos	frontend CPU 和探针时延升高	KPIRoot 第二场景
Pod Kill	paymentservice	PodChaos	Pod 身份替换、状态与内存序列变化	KPIRoot 补充场景
Network Delay	前端链路	NetworkChaos 配置	时延与错误率变化	扩展场景，未作为主结论

本组按照可控、可恢复和可标注三个原则设计故障场景。CPU Stress 的指标变化方向明确，适合验证根因定位；Pod Kill 更贴近实例故障，但 Kubernetes 的快速自愈会缩短异常窗口，因此实验同时保留 Pod 身份、Event 和 restart 等辅助证据。当前 AIOps 主线已完整验证 paymentservice CPU Stress，其他故障类型用于论文复现或作为扩展场景。

ChaosMesh KPI Collection Evidence

Item	Value
PodChaos status	Selected=True, AllInjected=True
Fault window	samples 45-74
Rows / metrics	120 / 16
Precision / Recall	0.4643 / 0.7429
F1	0.5714
Top signal	frontend_latency_ms

图 4 ChaosMesh PodChaos 注入状态及恢复结果摘要

3.3 数据组织与质量控制

数据采集按照 baseline、fault、recovery 三段组织。USAD 数据采用 timestamp、多个 KPI 列和可选 label 列；KPIRoot 数据除宽表 kpi_matrix.csv 外，还保留 metadata.yaml、series_labels.json 和 Prometheus 原始 query_range 结果。服务级指标对动态 Pod 名称进行稳定化映射，以防 Pod 重建造成列名变化。每组实验同时保留截图、CSV、JSON、Markdown 摘要与算法图表，便于从视觉结果回溯到原始数据。

本项目包含两条数据链路。用于 USAD 离线评价的 120 点 ChaosMesh 闭环数据，由前端 HTTP 探针与 Kubernetes API 同步采集，共包含 16 项请求和集群状态指标；端到端 AIOps 实时主线则通过 Prometheus 查询当前监控窗口，并由 Adapter 生成 USAD 与 KPIRoot 的 runtime 输入。两类数据均来自真实运行环境，但采集接口和用途不同。

PodChaos 在第 45 个采样点执行，实验将第 45~74 个采样点定义为故障影响观察窗口。由于 Kubernetes 会自动重建被杀死的 Pod，服务实际恢复时间可能早于标签窗口结束，恢复过程的短时波动也可能延续到窗口之外。这种标签边界与真实恢复过程的不完全重合，是 Precision 低于可控示例实验的重要原因之一。

数据集	样本/矩阵规模	KPI 数量	主要用途
USAD 可控示例数据	720 点	9	验证模型训练、阈值与评价流程
Online Boutique 真实 KPI	120 点	15	真实系统异常检测

数据集	样本/矩阵规模	KPI 数量	主要用途
ChaosMesh-USAD 闭环数据	120 点	16	实际 PodChaos 同步采集与检测
KPIRoot payment CPU	115 × 62	62 列	paymentservice CPU 根因定位
KPIRoot frontend CPU	68 × 60	60 列	frontend CPU 根因定位
KPIRoot payment Pod Kill	67 × 62	62 列	服务级 Pod Kill 根因定位

4 Selenium 与 JMeter 黑盒测试

4.1 Selenium 功能与兼容性测试

Selenium 自动化覆盖首页商品列表、商品详情和购物车等核心路径，并保存页面截图与耗时 JSON。基础测试在 Chrome 环境完成，补强实验通过统一多浏览器脚本在 Microsoft Edge 上复测相同用户路径，从而验证页面结构、链接跳转和购物车访问在不同 Chromium 浏览器中的兼容性。Edge 实测首页、商品页和购物车均通过，耗时分别为 1220.02 ms、197.04 ms 和 83.95 ms。本轮实验未配置 Firefox 运行环境，兼容性结果以已完成的 Chrome 和 Edge 测试为准。

浏览器	首页	商品页	购物车	结论
Microsoft Edge	1220.02 ms	197.04 ms	83.95 ms	全部 passed

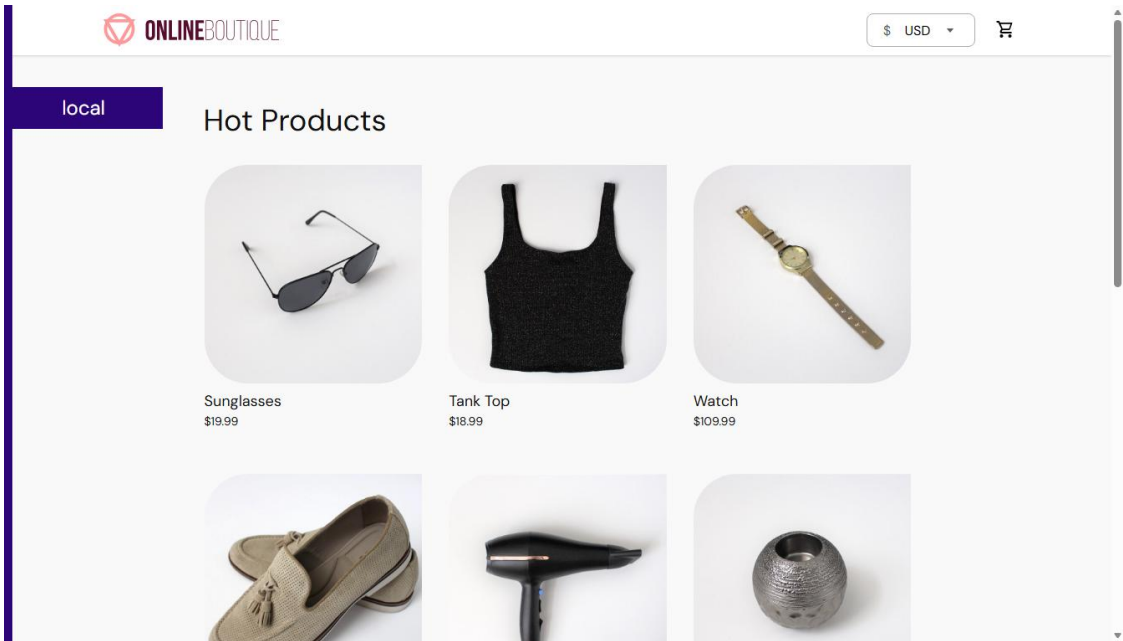


图 5 Selenium Edge 浏览器首页兼容性测试截图

4.2 JMeter 多并发性能测试

JMeter 测试计划持续访问首页、商品页和购物车页，补强实验使用 10、30、50 三档线程数，比较样本数、错误率、平均响应时间、P95 和最大响应时间。三档测试错误率均为 0，说明在本地实验负载下功能保持稳定；但平均响应时间从 40.20 ms 上升到 404.30 ms，P95 从 94 ms 上升到 902 ms，表明并发提高后排队和资源竞争显著增强。

测试在 Windows、Docker Desktop 与 Minikube 本地环境中进行，JMeter 版本为 5.6.3，通过 port-forward 访问 Online Boutique。每个线程循环 8 次，每次依次请求首页、商品页和购物车，因此样本数 = 并发线程数 × 8 × 3，分别得到 240、720 和 1200 个样本。这些耗时用于同一环境下的相对对比，不直接代表生产环境容量上限。

并发线程	样本数	错误率	平均响应	P95	最大响应
10	240	0.00%	40.20 ms	94 ms	164 ms
30	720	0.00%	180.22 ms	354 ms	977 ms
50	1200	0.00%	404.30 ms	902 ms	1344 ms

JMeter 10/30/50 Concurrency Summary

Threads	Samples	Error	Avg ms	P95 ms	Max ms
10	240	0.00%	40.20	94.00	164.00
30	720	0.00%	180.22	354.00	977.00
50	1200	0.00%	404.30	902.00	1344.00

图 6 JMeter 10/30/50 并发性能对比摘要

测试结论：零错误率只能说明请求成功，不代表性能不受影响。平均值、P95 和最大值随并发同步升高，才是容量变化的关键证据。

5 论文一：USAD 异常检测复现

5.1 方法原理

USAD（UnSupervised Anomaly Detection on Multivariate Time Series）面向 IT 监控中的多变量时间序列。其核心假设是：只使用正常阶段窗口训练重构网络后，模型能够学习 KPI 之间的正常联合模式；故障窗口偏离正常模式，因而产生更高重构误差。模型由共享编码器 E 和两个解码器 D1、D2 构成，通过双重重构目标增强正常与异常窗口的区分。本项目使用轻量 NumPy 版本保留共享编码器、双解码器和加权重构评分逻辑，以降低课程环境依赖。该实现聚焦论文核心机制，在网络规模和训练方式上与原论文完整深度学习实现存在差异。

1. 对 KPI 进行缺失值处理和标准化，将连续序列切分为固定长度滑动窗口。
2. 使用正常窗口训练共享编码器及两个解码器，学习正常状态下的联合表示。
3. 对所有窗口计算两个解码器的重构误差并形成异常分数。
4. 以正常训练分数的高分位数作为阈值，超过阈值的窗口判定为异常。
5. 将窗口预测映射回采样点，计算 Precision、Recall、F1，并按 KPI 汇总重构误差用于解释。

5.2 三层实验设计与结果

实验层次	样本	Precision	Recall	F1	意义
可控示例	720 点、9 KPI	0.5889	1.0000	0.7413	验证实现与评价链路
Online Boutique 真实数据	120 点、15 KPI	0.6071	0.9714	0.7473	验证真实系统迁移
ChaosMesh 真实闭环	120 点、16 KPI	0.4643	0.7429	0.5714	验证注入—采集—检测同链路

ChaosMesh 真实闭环由实际执行 PodChaos、同步采集 120 个真实 KPI 采样点并立即运行 USAD 构成。该结果的 F1 低于早期真实数据实验，说明更真实的故障窗口带来了恢复过程、网络抖动和标签边界不完全一致等问题。Recall 0.7429 表明多数异常被捕获，Precision 0.4643 则提示误报仍较明显。相较只在人工构造数据上获得更高指标，这一结果更能说明算法进入真实微服务环境后面临的阈值和标注挑战。

关键参数：本轮真实闭环采用以下固定配置，以便复现实验结果。

样本数	120	KPI 数量	16
采样间隔	1 s	故障观察窗口	45~74
滑动窗口	6	训练比例	0.35
训练轮数	120	检测阈值	0.222608

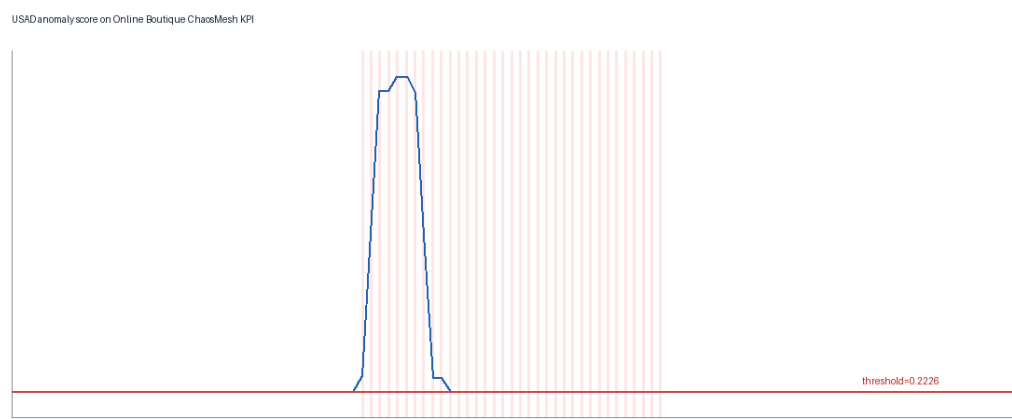


图 7 ChaosMesh 真实 KPI 数据的 USAD 异常分数与阈值

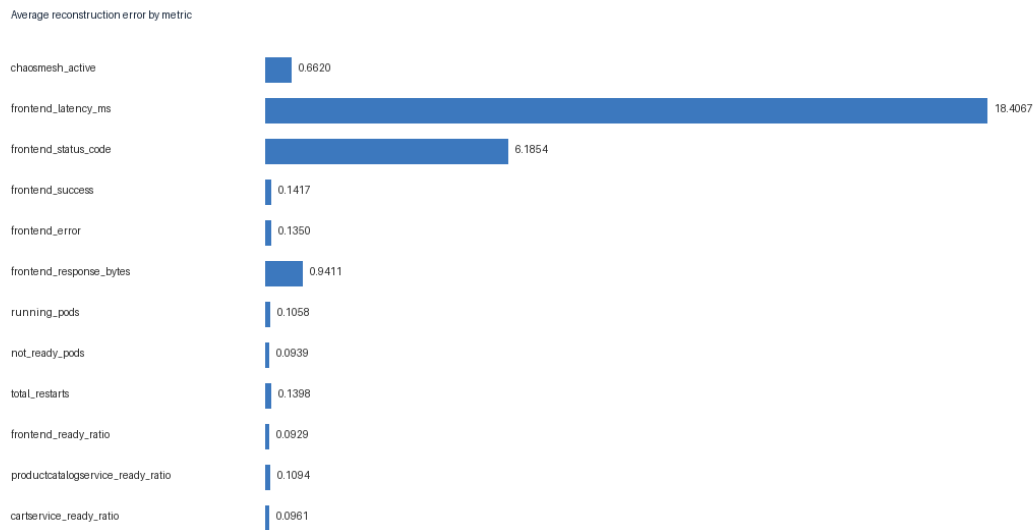


图 8 ChaosMesh 真实 KPI 的指标级平均重构误差

5.3 结果解释与局限

- 高召回说明 USAD 对故障造成的联合 KPI 偏移较敏感，适合作为异常发现层。
- 误报与正常训练样本不足、阈值分位数敏感、故障恢复阶段波动及窗口标签边界有关。
- 指标级重构误差能够给出候选异常 KPI，但不能直接证明因果关系，因此需要 KPIRoot 或在线证据进一步定位。
- 轻量 NumPy 实现降低了复现门槛，但与论文原始深度网络、训练轮数和工业数据规模存在差异。

6 论文二：KPIRoot 根因定位复现

6.1 问题定义与核心算法

KPIRoot 的输入前提与 USAD 不同：系统已经出现告警，需要从大量候选 KPI 中定位最可能驱动告警的根因指标。算法先对时间序列标准化，使用 PAA 降低序列长度，再通过 SAX 将连续数值

离散为符号序列；随后计算告警 KPI 与候选 KPI 在异常窗口内的 multiset-Jaccard 相似度，并使用 Granger 风格回归衡量候选 KPI 对告警 KPI 的时序解释能力。最终综合得分采用 $0.9 \times \text{similarity} + 0.1 \times \text{normalized causality}$ 。较高的相似度权重符合原论文设置，也更适合本项目较短、故障趋势较直接的课程数据。

步骤	参数/实现	作用
标准化	逐 KPI z-score，常数序列置零	消除量纲差异
PAA	paa_size=32	压缩时间序列并保留故障趋势
SAX	alphabet_size=9	将连续趋势转为可比较符号
相似度	SAX multiset-Jaccard	比较异常窗口形状
因果得分	Granger 风格，lag=2	估计候选 KPI 的时序先行性
综合排序	0.9 相似度 + 0.1 因果	输出根因候选 Top-K

6.2 Online Boutique 场景适配

原论文面向 Cloud H 的集群告警 KPI 与 VM 级 KPI，本项目将实体映射为 Online Boutique 的系统级告警指标和服务级候选指标。CPU Stress 场景使用 synthetic_total_cpu 或相应服务 CPU 作为告警关联依据；Pod Kill 场景由于服务快速重建，使用聚合内存和运行状态等稳定信号辅助定位。动态 Pod 名称被归并到服务名，从而将目标从“某个临时 Pod”提升为“哪个业务服务”。这种处理提高了服务级可解释性，但会弱化单 Pod 身份切换信息，属于明确的工程折中。

6.3 主实验结果

评价真值由故障注入对象确定：向 paymentservice 注入 CPU Stress 或 Pod Kill 时，paymentservice 即为预期根因服务；向 frontend 注入 CPU Stress 时，frontend 为预期根因服务。算法输出的 KPI 再映射到服务，并使用根因服务排名与 Hit@1/3/5 评价定位效果。服务排名为 1 表示最高优先级候选属于实际注入故障的服务。

故障场景	真实根因服务	Top-1 KPI	服务排名	Hit@1/3/5
paymentservice CPU Stress	paymentservice	cpu__paymentservice	1	True / True / True
frontend CPU Stress	frontend	cpu__frontend	1	True / True / True
paymentservice Pod Kill	paymentservice	memory__paymentservice	1	True / True / True

三组场景的真实根因服务排名均为 1。paymentservice CPU Stress 中 `cpu__paymentservice` 的相似度为 0.778，综合得分约 0.700；frontend CPU Stress 中 `cpu__frontend` 的相似度达到 1.000，综合得分约 0.902。Pod Kill 的 Top-1 指标为 `memory__paymentservice`，虽然不是 Pod 状态字段，但仍指向正确服务。其原因是 Kubernetes 快速重建目标 Pod，状态异常持续时间较短，而服务级内存序列保留了实例替换与恢复过程的变化。因此，该场景证明的是服务级根因定位有效，并不表示内存就是 Pod 被删除的直接物理原因。

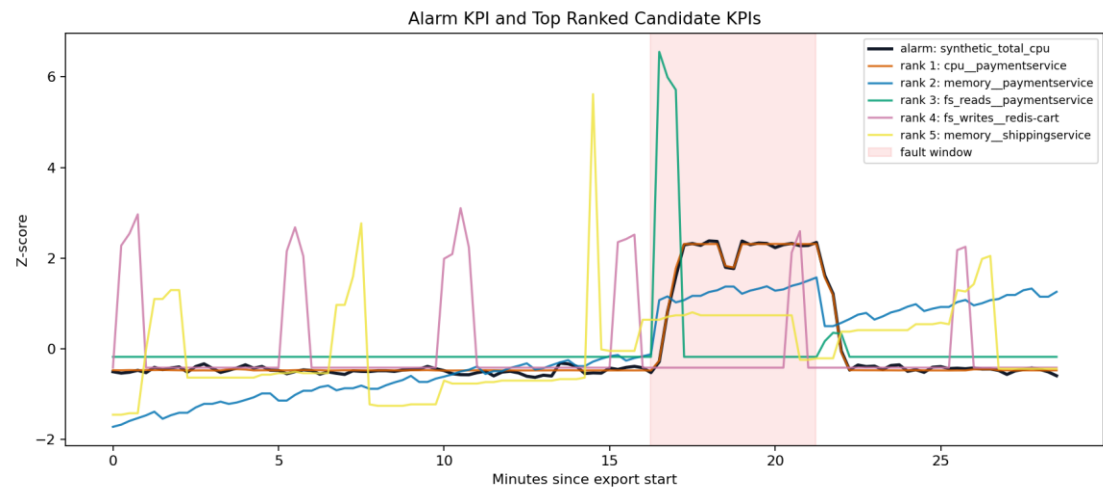


图 9 paymentservice CPU Stress：告警 KPI 与高排名候选 KPI 对比

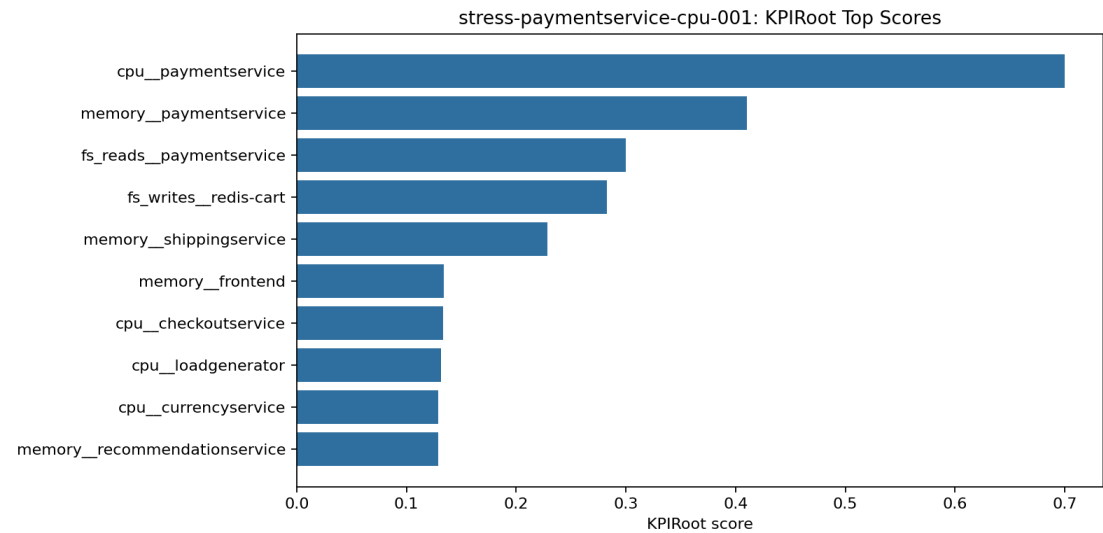


图 10 paymentservice CPU Stress：KPIRoot Top-K 综合得分

6.4 消融实验

场景	similarity_only	causality_only 根因排名	combined
paymentservice CPU Stress	排名 1，命中	排名 9，未命中	排名 1，命中

场景	similarity_only	causality_only 根因排名	combined
frontend CPU Stress	排名 1, 命中	排名 9, 未命中	排名 1, 命中
paymentservice Pod Kill	排名 1, 命中	排名 11, 未命中	排名 1, 命中

消融结果显示，单独使用 Granger 风格因果得分在三组短序列中均不稳定；相似度单项和综合方法均能命中真实服务。其原因不是因果分析在理论上无效，而是课程数据的故障窗口较短，PAA 后可用于回归的点数有限，同时 Prometheus 采样和 Kubernetes 自愈可能让非根因指标出现偶然先后关系。综合方法通过较高的相似度权重抑制了短序列因果噪声。

7 端到端实时 AIOps 智能运维

7.1 建设目标与系统架构

AIOps 部分不修改两篇论文项目内部算法，而是在其外部增加统一控制台和编排层。早期版本只读取已生成的 USAD/KPIRoot 输出，用于验证 Agent 汇总流程；当前主线已经升级为实时模式：每次故障实验后从 Prometheus 重新采集 Online Boutique 当前指标，生成 runtime 数据，真实执行 USAD 与 KPIRoot，再由 Agent 汇总 Kubernetes、Prometheus 和算法证据。旧离线能力仍保留在高级工具中，用于兼容和调试，但不作为最终演示主线。

端到端主链路：ChaosMesh 故障注入 → Prometheus 实时采集 → Adapter 生成统一 runtime 输入 → USAD 异常检测 → KPIRoot 根因排序 → Agent/LLM 证据融合 → 人工确认恢复。



图 11 AIOps 控制台系统总览及能力接入状态

模块	主要职责	真实执行状态
实时故障实验	生成并应用 ChaosMesh 实验对象，支持注入、清理和状态查看	CPU Stress 已完整验证
Prometheus 采集器	查询最近 N 分钟指标并生成 runtime CSV	真实执行
USAD 适配器	生成 canonical CSV 并调用 run_usad.py	execute 模式真实执行
KPIRoot 适配器	生成 phase2 场景目录并调用 kpiroot.cli	execute 模式真实执行
Agent	融合算法、Kubernetes 与 Prometheus 证据	真实生成报告
Ark LLM	生成结构化智能诊断总结	可选真实调用
恢复模块	输出重启、扩容等建议草案	仅 dry-run，不自动执行

7.2 推荐演示主线

- 6. 在“实时故障实验”选择 paymentservice 与 CPU 压力故障，设置持续 2 分钟、CPU Load 80、Workers 1。
- 7. 注入 StressChaos，等待 Prometheus 采集到 service_cpu_rate 的升高。
- 8. 进入“端到端 AIOps 诊断”，选择 execute USAD + KPIRoot，按 15 秒步长采集最近 5 分钟数据。
- 9. 系统生成新的 USAD/KPIRoot runtime 输入，分别执行异常检测和根因定位。
- 10. Agent 读取本轮输出和在线证据，给出 cpu_pressure_investigation / medium 结论。
- 11. 在“结果与报告中心”查看 Pipeline、USAD、KPIRoot、Agent 与可选 LLM 报告，最后清理故障。



图 12 实时故障实验页面：paymentservice CPU Stress 参数与诊断依据

7.3 实时流水线运行证据

2026 年 6 月 8 日 20:35 的成功运行报告显示：dry_run=False 仅表示外部算法不是模拟执行，execute_mode 为 execute_usad_kpiroot，data_source_mode 为 realtime_runtime。USAD 和 KPIRoot 的 executed、success 字段均为 True；KPIRoot 将 cpu_paymentservice 排在第一位，真实根因服务排名为 1；Prometheus service_cpu_rate 为 0.110536，高于系统设置的 0.05 CPU 压力阈值，Agent 因此输出 cpu_pressure_investigation，风险等级为 medium。

证据字段	值	说明
data_source_mode	realtime_runtime	使用本次 Prometheus 实时采集数据
execute_mode	execute_usad_kpiroot	两种算法均被编排执行
USAD executed / success	True / True	真实调用 run_usad.py 并生成输出
KPIRoot executed / success	True / True	真实调用 kpiroot.cli 并生成排序
KPIRoot Top-1	cpu_paymentservice	预期服务排名 1，Hit@1=True
service_cpu_rate	0.110536	超过阈值 0.05
Agent decision	cpu_pressure_investigation	进入 CPU 压力排查
risk_level	medium	需要人工复核，不自动恢复

口径说明：实时短窗口的监督评价指标受标签是否存在影响。报告将“算法执行成功”“异常分数越阈”和“监督 F1”分开描述，不以无标签 runtime 的 F1 代替真实闭环数据集上的评价结果。

>>

Deploy

AIOPS Agent 智能运维控制台

系统总览 实时故障实验 **端到端 AIOPS 诊断** 结果与报告中心 高级工具 项目架构说明

端到端 AIOPS 诊断

本页面会从 Prometheus 采集当前 Online Boutique 实时指标，生成 USAD/KPIRoot runtime 输入，并根据执行模式真实运行 USAD/KPIRoot，再由 Agent 综合生成诊断报告。推荐正式演示选择：execute USAD + KPIRoot。

当前目标服务
paymentservice

实验场景类型
CPU 压力故障（已完整验证）

该选项用于场景归档和 KPIRoot scenario 生成，不作为诊断结论；最终结论以 USAD/KPIRoot、Kubernetes、Prometheus 和 Agent 综合诊断为准。

CPU 压力故障是当前已完整验证场景，主要依赖 Prometheus service_cpu_rate。

duration_minutes
5

step_seconds
15

执行模式
dry-run，只采集数据并规划命令

为了安全，恢复动作保持 dry-run，不会自动执行真实恢复命令；推荐正式演示选择：execute USAD + KPIRoot。

USAD epochs
1

USAD window
5

USAD train ratio
0.70

KPIRoot scenario
realtime-paymentservice-cpu

KPIRoot alarm
paymentservice

☐ 启用 LLM 总结

运行端到端实时 AIOPS 诊断

查看最新端到端 AIOPS 报告

图 13 端到端 AIOPS 诊断参数页面（图中为安全 dry-run 配置界面）

7.4 Agent 与 Ark LLM 诊断

另一轮 20:32 的实时运行启用了火山方舟 Ark。该轮同样使用 realtime_runtime 并真实执行 USAD、KPIRoot；Prometheus service_cpu_rate 为 0.081961，USAD 检测到 1 个异常窗口，最大异常分数 2.0926，阈值 2.0887；KPIRoot Top-1 为 cpu_paymentservice。LLM 结合这些结构化证据生成“paymentservice 存在 CPU 压力异常”的总结，建议检查日志、Kubernetes Event、流量峰值和资源 request/limit，并明确所有重启、扩容操作均需人工确认。

证据来源	关键结果	在诊断中的作用
USAD	has_anomaly=True, anomaly_windows=1	证明当前窗口偏离正常模式
KPIRoot	top_service=paymentservice; top_metric=cpu_paymentservice	定位主要异常服务和指标
Prometheus	service_cpu_rate=0.081961	提供当前实时 CPU 证据

证据来源	关键结果	在诊断中的作用
Kubernetes	health_status=healthy	排除 CrashLoop、调度失败等结构性故障
Agent	cpu_pressure_investigation / medium	综合证据形成处置级别
Ark LLM	volcengine_ark, 执行成功	将结构化证据转为可读诊断总结

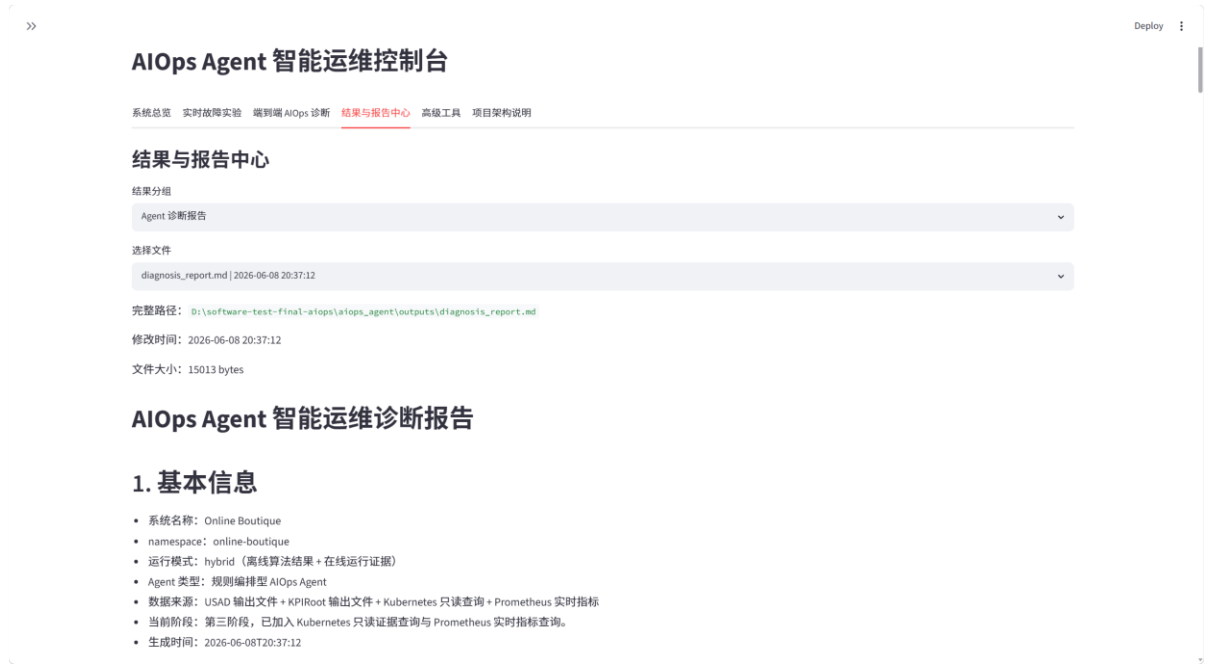


图 14 结果与报告中心：Agent 诊断报告的统一查看入口

7.5 安全边界

dry-run 的准确含义：dry-run 只保护恢复动作。故障注入、Prometheus 采集、USAD、KPIRoot、Agent 报告和可选 LLM 调用均可真实执行；系统不会自动执行 rollout restart、scale、delete Pod 等恢复命令。

- API Key 仅通过环境变量或当前 Streamlit 会话传入，不写入 config.json、源码或报告。
- 外部论文项目原始输出不被覆盖，runtime 结果统一写入 aiops_agent/runtime_outputs。
- 恢复建议以命令草案展示，必须由操作者核查服务状态和风险后手动执行。
- 当前完整验证场景为 paymentservice CPU Stress；内存、Pod Kill、网络延迟仅作为扩展框架。

8 两篇算法与 AIOps 的综合分析

8.1 USAD 与 KPIRoot 的互补关系

维度	USAD	KPIRoot	组合价值
核心问题	系统是否出现异常	异常由哪个 KPI/服务驱动	从发现走向定位
输入	多变量 KPI 窗口	告警 KPI + 候选 KPI	共享同一采集数据
输出	异常分数、阈值、异常窗口	Top-K 根因指标与服务排名	形成结构化诊断证据
优势	无监督、适合未知异常	排序可解释、适合运维定位	兼顾检测覆盖与解释性
局限	不能直接证明根因	依赖告警窗口和候选指标质量	由 Agent 与在线证据缓解

USAD 和 KPIRoot 不是相互替代的算法。USAD 的高召回适合在持续监控中发现偏离，KPIRoot 则在异常发生后将运维人员的搜索范围从数十条 KPI 缩小到少数服务和指标。AIOps 编排层进一步将两者与在线 CPU、内存、Pod、Event 证据结合：如果 USAD 告警但 KPIRoot 和实时指标不支持同一根因，系统应进入 manual_review；如果三类证据一致，则可给出更明确的调查方向。

8.2 结果一致性与冲突处理

- 12. 一致场景：USAD 异常、KPIRoot 指向 paymentservice CPU、Prometheus CPU 超阈值，输出 CPU 压力排查。
- 13. 算法与在线证据不一致：保留算法结果，但以当前 Prometheus 和 Kubernetes 状态决定风险等级。
- 14. Prometheus 查询为空：不使用旧值冒充当前状态，输出 manual_review 并提示等待采集窗口后重试。
- 15. 恢复后短时残留异常：允许 USAD 仍存在少量异常窗口，但若实时 CPU 已降低且集群健康，决策可恢复为 observe / low。

实验分析原则：本项目将算法结果、在线证据与运维决策分开记录，并通过原始数据、运行输出和诊断报告保持实验过程可追溯。

9 达成度、创新点与不足

9.1 课程目标达成度

评价维度	完成情况	结论
复杂微服务系统	Online Boutique 完成部署、访问与运维监控	达到第二档
微服务开发	新增 anomaly-detector 与 probe-exporter	达到第三档
两篇论文复现	USAD 异常检测 + KPIRoot 根因定位	满足基本要求
测试工具	Selenium 核心路径与 Edge; JMeter 三档并发	满足并补强
故障与数据闭环	ChaosMesh 注入、真实 KPI 采集、算法运行	完整闭环
智能体加分项	实时 AIOps Dashboard、Agent、可选 Ark LLM	完成加分项

9.2 项目创新点

- 将异常检测与根因定位两篇论文放入统一数据和演示链路，而不是分别呈现孤立结果。
- 完成实际 PodChaos、120 点 KPI 同步采集与 USAD 检测闭环，并分析真实环境下指标下降的原因。
- 通过 JMeter 并发矩阵和 Edge 复测扩展了基础黑盒测试，结论不局限于单负载或单浏览器。
- AIOps 主线按本次故障重新生成 runtime 输入并真实执行两种算法，避免将历史输出包装成实时结果。
- 以 dry-run 明确划分自动诊断与自动恢复边界，在展示智能化能力的同时控制集群修改风险。

9.3 局限与改进方向

当前局限	影响	改进方向
USAD 正常训练数据较短	阈值不稳定、误报偏高	延长基线采集，引入按业务时段分层阈值
轻量 USAD 非原论文完整深度实现	与论文数值不可直接横比	补充 PyTorch 原版训练与多随机种子实验
KPIRoot 短序列因果得分不稳定	causality_only 排名偏离	延长采样窗口，增加采样频率并检验显著性
Pod 名称服务级聚合	弱化单实例替换信息	保留 Pod 级 KPI、Event 和日志特征
AIOps 主要完整验证 CPU Stress	泛化结论有限	补充内存、网络延迟和级联故障的端到端验证
LLM 输出依赖外部接口	网络或额度可能影响演示	保留 deterministic fallback 与缓存示例

10 小组成员分工

小组采用“论文复现分组 + 集成负责人”的协作方式。两篇论文分别由对应成员完成阅读、实现和结果校核，王鼎同学负责在算法项目外部构建 AIOps 集成层。最终报告在保留个人工作边界的同时，将各部分按课程四阶段重新组织。

成员	学号	具体贡献
李响	2311467	负责 USAD 论文阅读与主体复现；完成可控示例和 Online Boutique 真实 KPI 的数据处理、模型训练与评价；负责 Online Boutique 部署、Prometheus/Grafana 监控、自研 anomaly-detector 和 probe-exporter 微服务、基础 Selenium/JMeter 测试以及 USAD 项目主体文档。
陈祖名	2313163	完成 ChaosMesh 真实闭环：实际执行 PodChaos、同步采集 120 个 KPI 采样点并运行 USAD，生成 CSV、图表和说明；新增 JMeter 10/30/50 并发矩阵脚本及汇总；新增多浏览器脚本并完成 Edge 首页、商品页、购物车兼容性测试。
王新杰	2311901	参与 ISSRE24-KPIRoot 论文阅读；负责环境部署、Prometheus/Grafana 监控采集、ChaosMesh 故障数据整理、KPIRoot 算法复现、三场景主实验与消融实验，并完成对应报告主体。
王子祺	2312385	参与 KPIRoot 论文阅读和复现方案讨论；负责实验结果校核、算法适配逻辑复核、报告结构与表述整理。
王鼎	2213002	负责 AIOps 智能运维控制台；实现 Dashboard 故障实验、端到端诊断和结果中心；完成 Prometheus 实时采集、runtime 数据转换、USAD/KPIRoot 真实调用、Agent 综合诊断、Ark LLM 可选总结、多故障框架与 dry-run 安全机制。

11 总结

本项目完成了从云原生微服务部署到智能运维诊断的完整课程实践。Online Boutique 提供了真实的多服务运行环境，Prometheus、Grafana 和 ChaosMesh 构成可观测与故障实验基础，Selenium 和 JMeter 从功能、兼容性和性能角度验证系统质量。USAD 复现证明多变量重构误差能够捕获故障窗口，KPIRoot 复现证明形状相似度与因果得分组合可以在三组故障中定位正确服务。最终的 AIOps 控制台将二者与在线证据整合，形成可重复执行的“故障注入—实时采集—异常检测—根因定位—综合诊断—人工确认恢复”链路。

从实验结果看，工程真实性与算法指标之间存在必要张力：真实 ChaosMesh 数据上的 USAD 指标低于可控数据，短序列上的 KPIRoot 因果项也不稳定，但这些差异恰恰揭示了监控窗口、标签边界、采样频率和 Kubernetes 自愈机制对算法应用的影响。通过保留原始数据、消融结果、在线证据和安全边界，项目不仅展示了功能“能运行”，也说明了结果“为什么可信、何时不可信以及如何改进”。

参考文献

- [1] J. Audibert, P. Michiardi, F. Guyard, S. Marti, M. A. Zuluaga. USAD: UnSupervised Anomaly Detection on Multivariate Time Series. KDD, 2020.
- [2] W. Gu et al. KPIRoot: Efficient Monitoring Metric-based Root Cause Localization in Large-scale Cloud Systems. ISSRE, 2024.
- [3] GoogleCloudPlatform. Online Boutique / Microservices Demo.
- [4] Chaos Mesh Documentation. Cloud-native Chaos Engineering Platform.
- [5] Prometheus Documentation. Monitoring System and Time Series Database.
- [6] Grafana Documentation. Observability Dashboards and Visualization.
- [7] Apache JMeter User Manual.
- [8] Selenium Documentation.

项目仓库

USAD 与测试补强仓库: <https://github.com/asjdhgs/software-testing-large-homework>

KPIRoot 复现仓库: <https://github.com/Sinclair987/software-testing-final-project>

项目完整展示仓库: <https://github.com/sevenpeek/software-testing-aiops-final-project>

AIOps 集成代码、runtime_data、runtime_outputs、Agent 报告和 Dashboard 运行证据已统一收录于项目完整展示仓库。

附录 A 关键运行命令

任务	命令
USAD 基础复现	python src/run_usad.py --input data/sample_kpi_metrics.csv --out outputs
ChaosMesh-USAD 闭环	powershell -File scripts/run_chaosmesh_kpi_usad.ps1
JMeter 并发矩阵	powershell -File tests/jmeter/run_jmeter_matrix.ps1
Selenium 多浏览器	powershell -File tests/selenium/run_selenium_matrix.ps1
KPIRoot 批量运行	python -m kpiroot.cli --phase2-dir data/phase2 --output-dir data/phase4/kpiroot
AIOps 实时主线	python aiops_agent/realtime_pipeline_agent.py --config aiops_agent/config.json --duration-minutes 5 --step-seconds 15 --execute-external --usad-epochs 1 --usad-window 5 --usad-train-ratio 0.7 --kpiroot-scenario realtime-paymentservice-cpu --kpiroot-alarm paymentservice
Dashboard	streamlit run aiops_agent/dashboard_app.py

附录 B 报告证据索引

证据类别	代表文件/目录	支撑结论
USAD 数据与输出	data/; outputs_online_boutique_chaosmesh/	真实 KPI 检测指标与异常曲线
测试结果	outputs_jmeter_matrix/; outputs_selenium_matrix/	并发性能与 Edge 兼容性
KPIRoot 数据与输出	data/phase2/; data/phase4/kpiroot/	三场景 Top-K 与消融实验
AIOps 实时数据	aiops_agent/runtime_data/	本次 Prometheus 采集及算法输入
AIOps 实时输出	aiops_agent/runtime_outputs/	算法执行状态、排序、Pipeline 与 LLM 总结
Agent 报告	aiops_agent/outputs/diagnosis_report.md	综合诊断、风险等级和恢复建议